

Lucrare de laborator : 3

ASP.net Programing

C#

1. Introducere

ASP.NET este o platformă de dezvoltare web care oferă un model de programare, o infrastructură software complexă și diverse servicii necesare pentru a construi aplicații web robuste pentru PC, precum și pentru dispozitive mobile. ASP.NET funcționează pe lângă protocolul HTTP și utilizează comenzile și politicile HTTP pentru a stabili o comunicare și o cooperare bilaterală între browser și server. ASP.NET face parte din platforma Microsoft .Net. Aplicațiile ASP.NET sunt coduri compilate, scrise utilizând componente extensibile și reutilizabile sau obiecte prezente în cadrul .Net. Aceste coduri pot utiliza întreaga ierarhie a clasei în cadrul .Net.

Codurile aplicațiilor ASP.NET pot fi scrise în oricare dintre următoarele limbi:

- C#
- Visual Basic.Net
- Jscript
- J#

ASP.NET este folosit pentru a produce aplicații web interactive, bazate pe date, pe internet. Se compune dintr-un număr mare de comenzi, cum ar fi casetele de text, butoanele și etichetele pentru asamblarea, configurarea și manipularea codului pentru a crea pagini HTML.

1.1. Modelul de Formulare Web ASP.NET

Formularele web ASP.NET prelungesc modelul de interacțiune cu aplicațiile web. Browser-ul transmite un formular web către serverul web, iar serverul returnează o pagină completă de marcare sau o pagină HTML ca răspuns. Toate activitățile utilizatorilor din partea clientului sunt redirecționate către server pentru procesare statală. Serverul procesează rezultatele acțiunilor clientului și declanșează reacțiile. Acum, HTTP este un protocol stateless. Cadrul ASP.NET ajută la stocarea informațiilor privind starea aplicației, care constă în:

- Starea paginii
- Starea sesiunii

Starea paginii este starea clientului, adică conținutul câmpurilor de intrare diferite din formularul web. Starea sesiunii este informația colectivă obținută de la diverse pagini pe care utilizatorul a vizitat și a lucrat cu, adică starea generală a sesiunii. Pentru a elimina conceptul, să luăm un exemplu de coș de cumpărături. Utilizatorul adaugă articole într-un coș de cumpărături. Elementele sunt selectate dintr-o pagină, spun pagina de articole, iar totalul elementelor colectate și prețul sunt afișate pe o altă pagină, de exemplu, pagina de coș. Numai HTTP nu poate urmări toate informațiile provenite din diverse pagini. Starea sesiunii ASP.NET și infrastructura de pe partea serverului urmăresc informațiile colectate la nivel global în timpul unei sesiuni.

Rulația ASP.NET efectuează starea paginii către și de la server în întreaga solicitare a paginii în timp ce generează coduri runtime ASP.NET și încorporează starea componentelor de pe server în câmpuri ascunse. În acest fel, serverul devine conștient de starea generală a aplicației și funcționează într-un mod conectat pe două nivele.

1.2. Modelul de componente ASP.NET

Modelul de componentă ASP.NET furnizează diferite blocuri de construcție a paginilor ASP.NET. Practic este un model de obiect care descrie:

- Pe partea de server conține aproape toate elementele HTML sau tag-uri precum <form> sau <input>
- Controalele serverului, care ajută la dezvoltarea interfeței complexe de utilizator. De exemplu, controlul Calendar sau controlul Gridview.

ASP.NET este o tehnologie care funcționează pe cadrul .Net care conține toate funcționalitățile legate de web. Cadrul .Net este alcătuit dintr-o ierarhie orientată pe obiecte. O aplicație web ASP.NET este formată din pagini. Când un utilizator solicită o pagină ASP.NET, IIS delegă procesarea paginii în sistemul de rulare ASP.NET. Runtime-ul ASP.NET transformă pagina .aspx într-o instanță a unei clase, care moștenește din pagina de clasă de bază a cadrului .Net. Prin urmare, fiecare pagină ASP.NET este un obiect și toate componentele sale, adică, controalele de pe server sunt, de asemenea, obiecte.

1.3. Componentele framework-ului .NET 3.5

Înainte de a merge la următoarea sesiune pe Visual Studio.Net, să trecem la diferitele componente ale cadrului .Net 3.5. Tabelul următor descrie componentele cadrului .Net 3.5 și sarcinile pe care le efectuează:

Common Language Runtime or CLR	Windows Workflow Foundation (WF)
.Net Framework Class Library	Windows Presentation Foundation
Common Language Specification	Windows Communication Foundation (WCF)
ADO.NET	LINQ
Windows Forms	Metadata and Assemblies
ASP.NET and ASP.NET AJAX	

ASP.NET oferă un strat de abstractizare pe partea de sus a HTTP pe care sunt construite aplicațiile web. Oferă entități la nivel înalt, cum ar fi clasele și componentele dintr-o paradigmă orientată spre obiect. Instrumentul cheie de dezvoltare pentru dezvoltarea aplicațiilor ASP.NET și a front-end-urilor este Visual Studio. În acest tutorial, lucrăm cu Visual Studio 2015. Visual Studio

este un mediu integrat de dezvoltare pentru scrierea, compilarea și depanarea codului. Acesta oferă un set complet de instrumente de dezvoltare pentru construirea de aplicații web ASP.NET, servicii web, aplicații desktop și aplicații mobile.

1.4. Ciclul de viața ASP.NET

Ciclul de viață ASP.NET specifică cum:

- Aplicația și paginile sale sunt instanțiate și procesate
- ASP.NET compilează paginile dynamic
- ASP.NET procesează paginile pentru a produce ieșiri dinamice

Ciclul de viață ASP.NET ar putea fi împărțit în două grupuri:

- Ciclul de viață al aplicației
- Ciclul de viață al paginii

Ciclul de viață al aplicației are următoarele etape:

Utilizatorul face o solicitare de accesare a unei resurse de aplicație, o pagină. Browser-ul trimite această cerere către serverul web. O conductă unificată primește prima solicitare și are loc următoarele evenimente: Se creează un obiect din clasa `ApplicationManager`. Un obiect al clasei `HostingEnvironment` este creat pentru a oferi informații cu privire la resurse. Elementele de nivel superior din aplicație sunt compilate. Elementele de răspuns sunt create. Obiectele aplicației, cum ar fi `HttpContext`, `HttpRequest` și `HttpResponse`, sunt create și inițializate. O instanță a obiectului `HttpApplication` este creată și atribuită solicitării. Cererea este procesată de clasa `HttpApplication`. Diferite evenimente sunt generate de această clasă pentru procesarea cererii.

Ciclul de viață al paginii ASP.NET

Când se solicită o pagină, aceasta este încărcată în memoria serverului, procesată și trimisă în browser. Apoi este descărcat din memorie. La fiecare dintre aceste etape, sunt disponibile metode și evenimente care ar putea fi suprimate în funcție de necesitatea aplicării. Cu alte cuvinte, puteți scrie propriul cod pentru a suprascrie codul implicit. Clasa `Page` creează un arbore ierarhic al tuturor comenzilor de pe pagină. Toate componentele de pe pagină, cu excepția directivelor, fac parte din acest arbore de control. Puteți vedea arborele de control adăugând `trace = "true"` la directiva de pagină. Vom acoperi directivele de pagină și urmăm în conformitate cu "directivele" și "tratarea evenimentelor".

Fazele ciclului de viață a paginii sunt:

- Inițializarea
- Instalarea comenzilor de pe pagină
- Restaurarea și întreținerea statului
- Executarea codurilor de gestionare a evenimentelor
- Redarea paginilor

Înțelegerea ciclului paginii ajută la scrierea de coduri pentru a face anumite lucruri specifice în orice etapă a ciclului de viață al paginilor. De asemenea, ajută la scrierea de comenzi personalizate și la inițializarea acestora la momentul potrivit, să le populeze proprietățile cu date de stare și să execute codul comportamentului de control.

Următoarele sunt diferitele etape ale unei pagini ASP.NET:

Cerere de pagină - Când ASP.NET primește o solicitare de pagină, decide dacă să analizeze și să compileze pagina sau ar exista o versiune stocată în pagină a paginii; în consecință, răspunsul este trimis.

Pornirea ciclului de viață a paginii - În această etapă, sunt setate obiectele Cerere și Răspuns. Dacă cererea este o cerere veche sau post înapoi, proprietatea `IsPostBack` a paginii este setată la `true`. Proprietatea `UIC` a paginii este de asemenea stabilită.

Inițializare pagină - În această etapă, comenzile de pe pagină sunt atribuite ID unic prin setarea proprietății `UniqueID` și temele sunt aplicate. Pentru o nouă solicitare, sunt încărcate date postback și proprietățile de control sunt readuse la valorile de stare.

Încărcarea paginii - În această etapă, proprietățile de control sunt setate utilizând valorile de stare a vizualizării și a stării de control.

Validare - Se numește metoda de validare a controlului de validare, iar după executarea ei cu succes, proprietatea `IsValid` a paginii este setată la `true`.

Întârzierea evenimentului postback - Dacă cererea este un postback (veche solicitare), se face apelul pentru tratarea evenimentului asociat.

Redarea paginilor - În această etapă, se afișează starea pentru pagină și toate comenzile sunt salvate. Pagina cheamă metoda `Render` pentru fiecare comandă, iar ieșirea de randare este înscrisă în clasa `OutputStream` a proprietății de răspuns a paginii.

Descărcare - Pagina redată este trimisă clientului, iar proprietățile paginii, precum răspunsul și solicitarea, sunt descărcate și toate curățările efectuate.

1.5. ASP.NET Evenimente de ciclu de viață a paginilor

În fiecare etapă a ciclului de viață al paginii, pagina ridică unele evenimente care pot fi codificate. Un manipulator de evenimente este, în esență, o funcție sau o subrutină, legată de eveniment, folosind attribute declarative, cum ar fi `OnClick` sau handler.

Următoarele sunt evenimentele privind ciclul de viață al paginilor:

- **PreInit** - `PreInit` este primul eveniment din ciclul de viață al paginilor. Verifică proprietatea `IsPostBack` și determină dacă pagina este o postback. Setează temele și paginile de master, creează controale dinamice și primește și stabilește valori ale proprietăților profilului.

Acest eveniment poate fi manipulat prin supraîncărcarea metodei `OnInit` sau crearea unui handler `OnInit`.

- `Init` - `Init` eveniment inițializează proprietatea de control și arborele de control este construit. Acest eveniment poate fi manipulat prin suprasolicitarea metodei `OnInit` sau crearea unui handler `OnInit`.
- `InitComplete` - Evenimentul `InitComplete` permite urmărirea stării de vizualizare. Toate comenzile activează urmărirea stării de vizualizare.
- `LoadViewState` - evenimentul `LoadViewState` permite încărcarea informațiilor de stare a vizualizării în controale.
- `LoadPostData` - În această fază, conținutul tuturor câmpurilor de intrare sunt definite cu eticheta `<form>` sunt procesate.
- `PreLoad` - `PreLoad` are loc înainte de încărcarea datelor postare înapoi în controale. Acest eveniment poate fi gestionat prin suprasolicitarea metodei `OnInit` sau crearea unui handler `OnInit`.
- `Încărcare` - evenimentul `Încărcare` este ridicat pentru prima pagină și apoi recursiv pentru toate controalele pentru copii. Comenzile din arborele de control sunt create. Acest eveniment poate fi manipulat prin supraîncărcarea metodei `OnLoad` sau crearea unui handler `OnLoad`.
- `LoadComplete` - Procesul de încărcare este finalizat, sunt executate comenzile pentru controlul evenimentului și are loc validarea paginii. Acest eveniment poate fi manipulat prin supraîncărcarea metodei `OnLoadComplete` sau crearea unui handler `OnLoadComplete`.
- `PreRender` - evenimentul `PreRender` apare chiar înainte ca ieșirea să fie redată. Prin manipularea acestui eveniment, paginile și comenzile pot efectua orice actualizare înainte ca ieșirea să fie redată.
- `PreRenderComplete` - Întrucât evenimentul `PreRender` este declanșat recursiv pentru toate comenzile pentru copii, acest eveniment asigură finalizarea fazei de pre-redare.
- `SaveStateComplete` - Stocul de control al paginii este salvat. Personalizarea, starea controlului și vizualizarea informațiilor de stare sunt salvate. Marcajul HTML este generat. Această etapă poate fi gestionată prin suprascrierea metodei `Render` sau crearea unui handler `OnRender`.
- `Unload` - Faza `Unload` este ultima fază a ciclului de viață al paginii. Aceasta ridică evenimentul `Unload` pentru toate controalele recursiv și, în sfârșit, pentru pagina în sine. Finalizarea curățării se face și toate resursele și referințele, cum ar fi conexiunile bazei de date, sunt eliberate. Acest eveniment poate fi manipulat prin modificarea metodei `OnUnload` sau crearea unui handler `OnUnload`.

2. Exemplu aplicatie Web Asp.net:

HTTP nu este doar pentru servirea paginilor web. HTTP este, de asemenea, o platformă puternică pentru crearea API-urilor care expun servicii și date. HTTP este simplu, flexibil și omniprezent. Aproape orice platformă despre care vă puteți gândi are o bibliotecă HTTP, astfel încât serviciile HTTP pot ajunge la o gamă largă de clienți, inclusiv browsere, dispozitive mobile și aplicații desktop tradiționale. ASP.NET Web API este un cadru pentru construirea API-urilor web pe partea de sus a .NET Framework. În acest tutorial, veți utiliza API Web ASP.NET pentru a crea un API web care returnează o listă de produse. :

Toate Produsele

- Supa de pui : \$1
- Yo-yo: \$3.75
- Ciocan: \$16.99

Cauta dupa Id:

Figura 3.1: Interfata Aplicatie

Porniti Visual Studio : File – New - New project(ProductsApp)

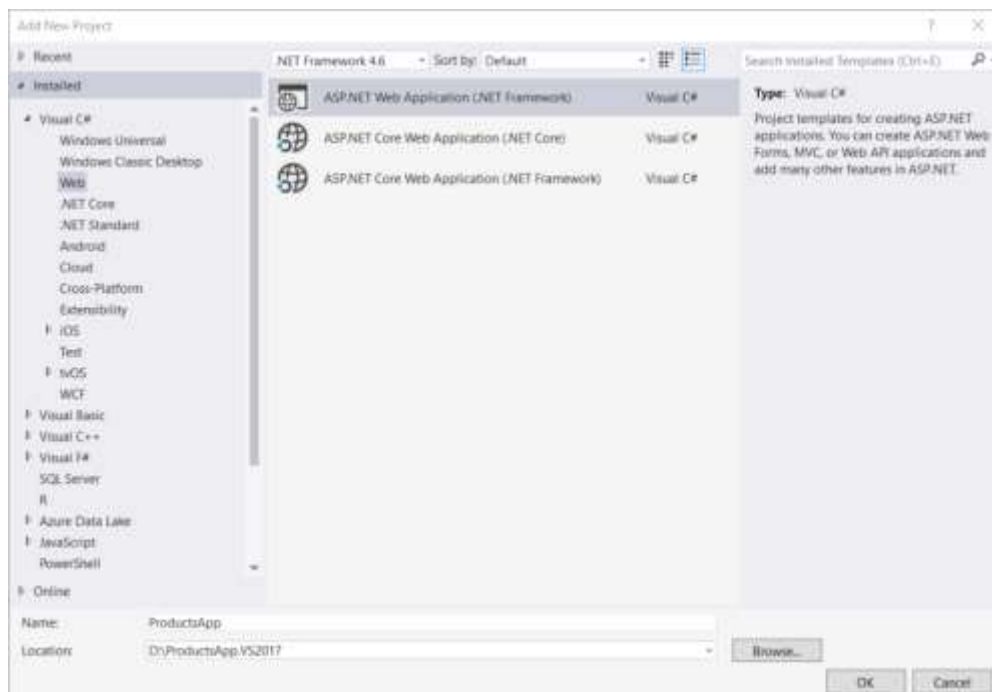


Figura 3.2: Template

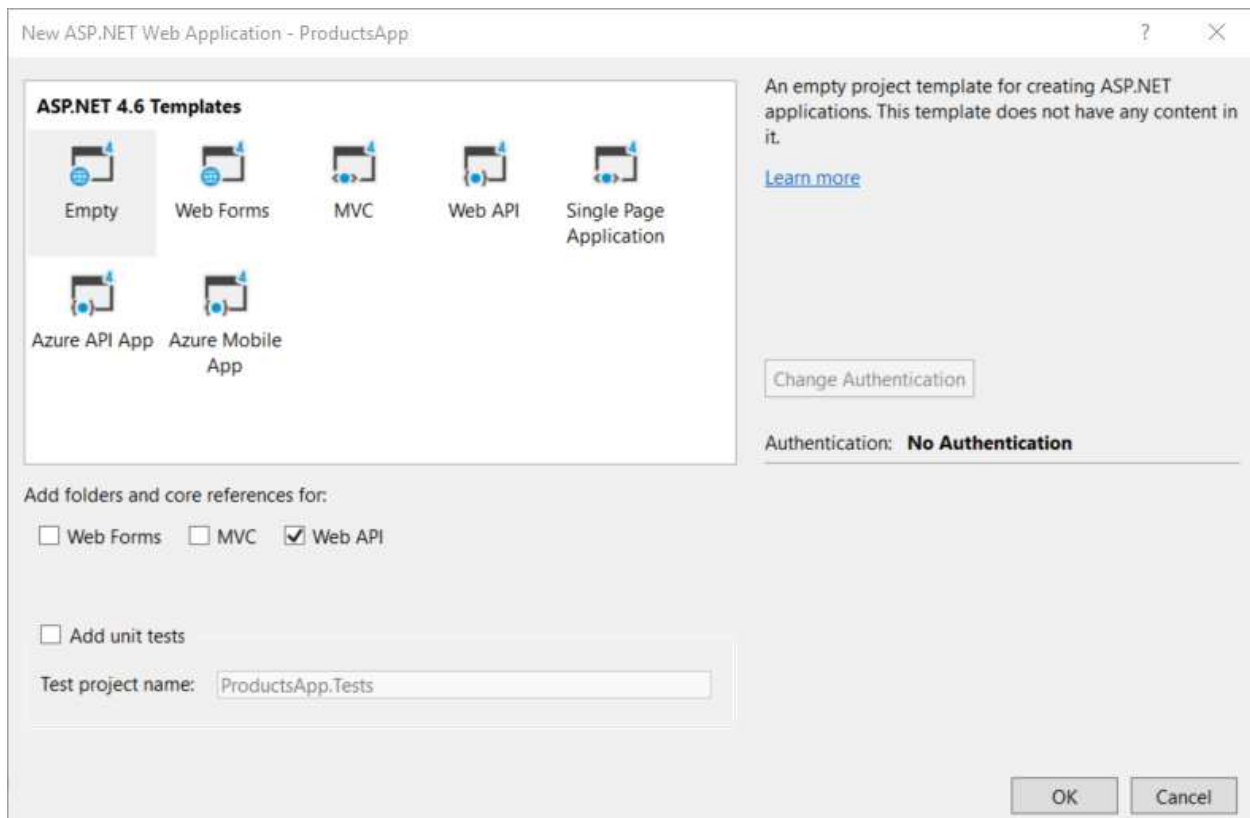


Figura 3.3: New ASP.NET Project

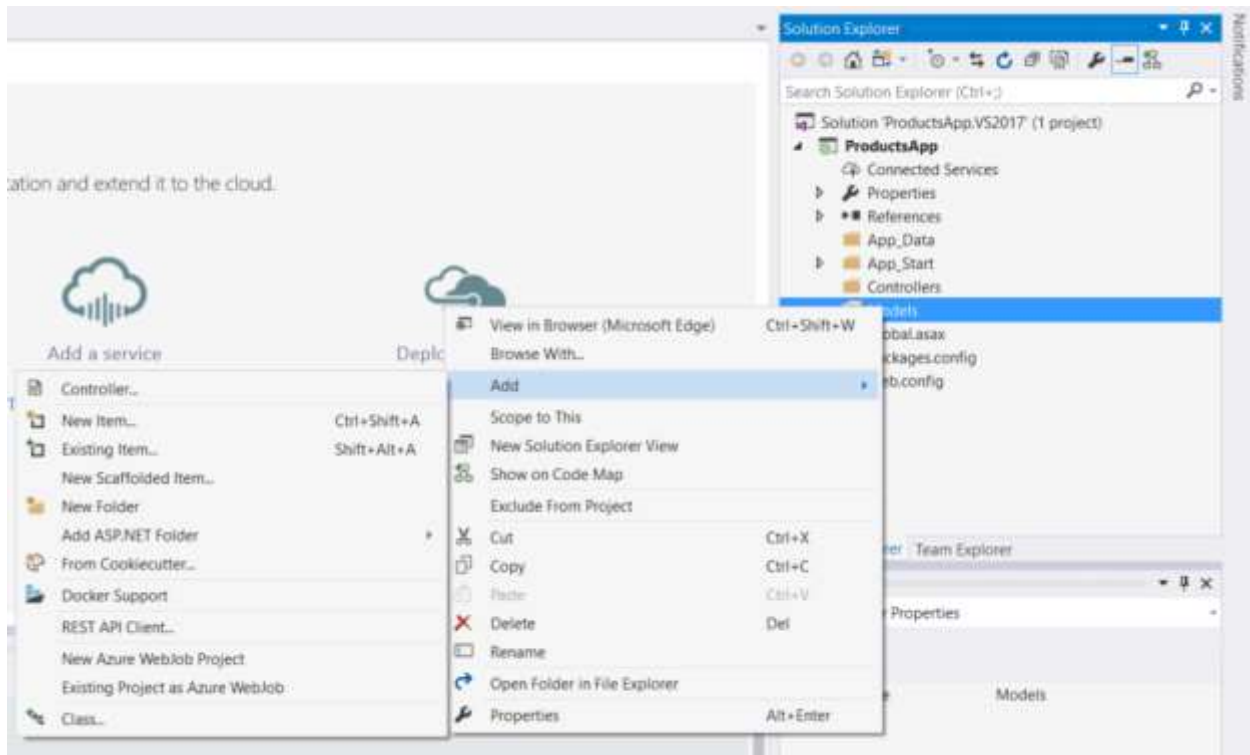


Figura 3.4: Adaugare a unui Model

Se va numi clasa Product(Produs):

```
C#

namespace ProductsApp.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Category { get; set; }
        public decimal Price { get; set; }
    }
}
```

Adaugare Controller :

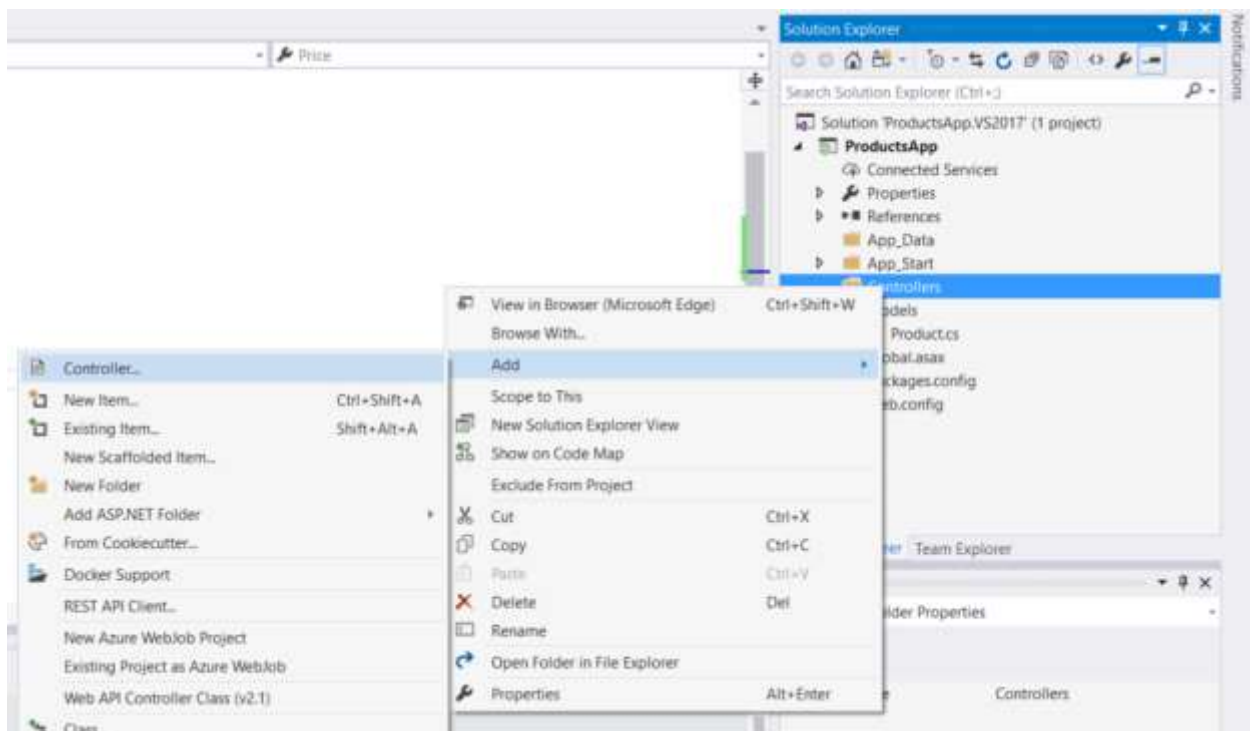


Figura 3.5: Adaugare a unui Controller

In **Add Scaffold** dialog, selecteaza **Web API Controller - Empty**. Click **Add**.



The scaffolding crează un fișier numit ProductsController.cs din folderul Controlere.

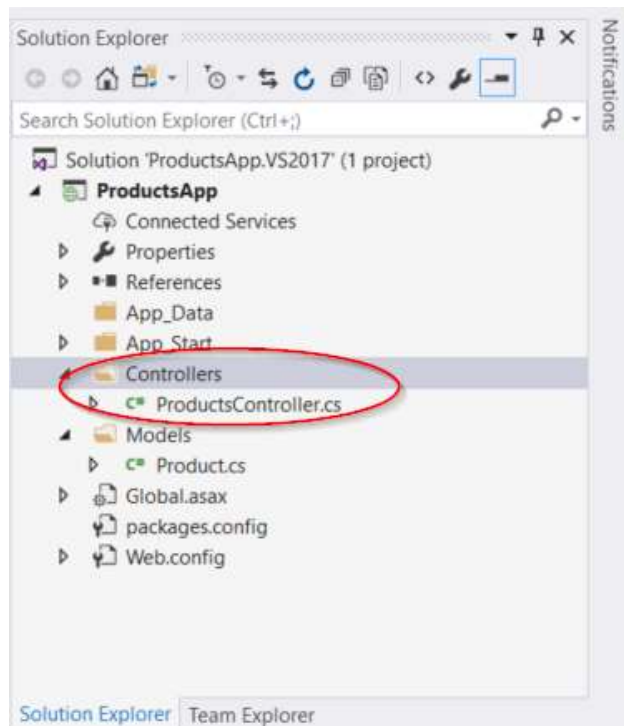


Figura 3.6: ProductsController.cs

```
using ProductsApp.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Web.Http;

namespace ProductsApp.Controllers
{
    public class ProductsController : ApiController
    {
        Product[] products = new Product[]
        {
            new Product { Id = 1, Name = "Tomato Soup", Category = "Groceries", Price = 1 },
            new Product { Id = 2, Name = "Yo-yo", Category = "Toys", Price = 3.75M },
            new Product { Id = 3, Name = "Hammer", Category = "Hardware", Price = 16.99M }
        };

        public IEnumerable<Product> GetAllProducts()
        {
            return products;
        }

        public IHttpActionResult GetProduct(int id)
        {
            var product = products.FirstOrDefault((p) => p.Id == id);
            if (product == null)
            {
                return NotFound();
            }
            return Ok(product);
        }
    }
}
```

Pentru a păstra exemplul simplu, produsele sunt stocate într-o matrice fixă în interiorul clasei de controler. Desigur, într-o aplicație reală, ați interoga o bază de date sau o altă sursă de date externă.

Controlorul definește două metode care returnează produsele:

Metoda GetAllProducts returnează întreaga listă de produse ca tip IEnumerable <Product>.

Metoda GetProduct caută un singur produs prin ID-ul său.

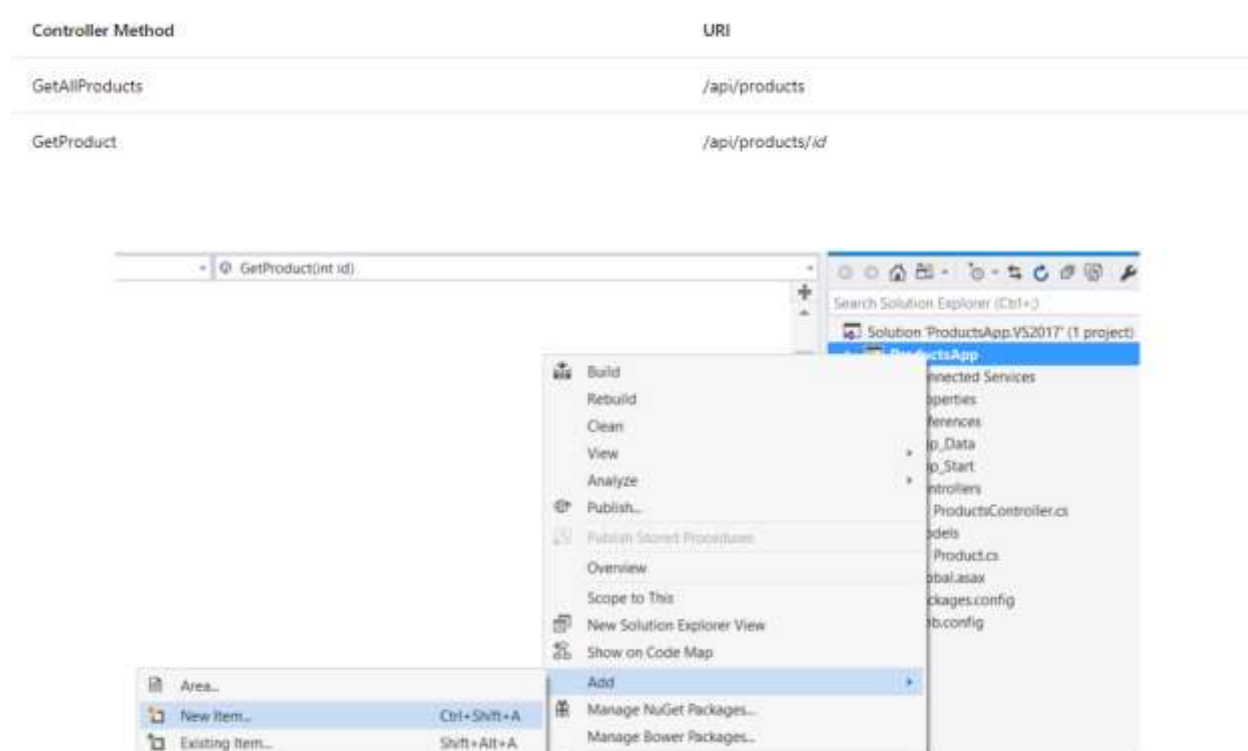


Figura 3.7: Adaugare new item

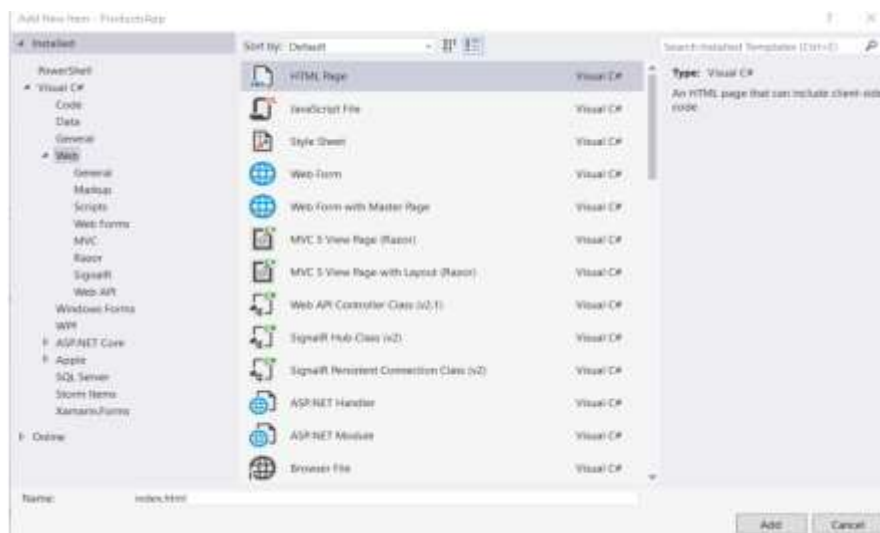


Figura 3.7: Adaugare new html page – index.xhtml

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Product App</title>
</head>
<body>

  <div>
    <h2>All Products</h2>
    <ul id="products" />
  </div>
  <div>
    <h2>Search by ID</h2>
    <input type="text" id="prodId" size="5" />
    <input type="button" value="Search" onclick="find();" />
    <p id="product" />
  </div>

<script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-2.0.3.min.js"></script>
<script>
  var uri = 'api/products';

  $(document).ready(function () {
    // Send an AJAX request
    $.getJSON(uri)
      .done(function (data) {
        // On success, 'data' contains a list of products.
        $.each(data, function (key, item) {
          // Add a list item for the product.
          $('<li>', { text: formatItem(item) }).appendTo($('#products'));
        });
      });

    function formatItem(item) {
      return item.Name + ': $' + item.Price;
    }

    function find() {
      var id = $('#prodId').val();
      $.getJSON(uri + '/' + id)
        .done(function (data) {
          $('#product').text(formatItem(data));
        })
        .fail(function (jqXHR, textStatus, err) {
          $('#product').text('Error: ' + err);
        });
    }
  });
</script>
</body>
</html>

```

Figura 3.8: cod – index.xhtml

```

HTML
$(document).ready(function () {
  // Send an AJAX request
  $.getJSON(apiUrl)
    .done(function (data) {
      // On success, 'data' contains a list of products.
      $.each(data, function (key, item) {
        // Add a list item for the product.
        $('<li>', { text: formatItem(item) }).appendTo($('#products'));
      });
    });
});

```

Figura 3.9: cod – get lista de produse

JavaScript

```
function find() {  
    var id = $('#prodId').val();  
    $.getJSON(apiUrl + '/' + id)  
        .done(function (data) {  
            $('#product').text(formatItem(data));  
        })  
        .fail(function (jqXHR, textStatus, err) {  
            $('#product').text('Error: ' + err);  
        });  
}
```

Figura 3.10: cod – get lista de produse dupa id

Rularea Aplicatiei :

Toate Produsele • Supa de pui : \$1 • Yo-yo: \$3.75 • Ciocan: \$16.99 Cauta dupa Id: 2 Cauta Yo-yo: \$3.75	Toate Produsele • Supa de pui : \$1 • Yo-yo: \$3.75 • Ciocan: \$16.99 Cauta dupa Id: 1 Cauta Supa de pui : \$1	Toate Produsele • Supa de pui : \$1 • Yo-yo: \$3.75 • Ciocan: \$16.99 Cauta dupa Id: 3 Cauta Ciocan: \$16.99
--	--	--

3. Teme de laborator :

- 3.1. Implementati aplicatia de mai sus
- 3.2. Adaugati inca 10 produse.

4. Bibliografie :

- 4.1. <https://docs.microsoft.com/en-us/aspnet/web-api/overview/getting-started-with-aspnet-web-api/tutorial-your-first-web-api>

